

Techniques for Architecting GPU Register File

Sparsh Mittal

IIT Roorkee, India



Register file (RF) size in CPU and GPU

- RF is required to hold thread-context.
- CPUs use small RF and much larger caches to hide latency of single thread
- GPUs use large number of threads and switch between them to hide memory latency. Thread-count is high => RF size required is also high
- GPU RF size is even greater than L1/L2 caches

Trends: RF size on GPUs is increasing

Architectural parameters of recent NVIDIA GPUs

	Architecture	Compute Capability	L1 size (KB)	L2 size (KB)	RF size (KB)	# of SMs	Total RF size (KB)
G80	Tesla	1.0	None	None	32	16	512
GT200	Tesla	1.3	None	None	64	30	1920
GF100	Fermi	2.0	48	768	128	16	2048
GK110	Kepler	3.5	48	1536	256	15	3840
GK210	Kepler	3.7	48	1536	512	15	7680
GM204	Maxwell	5.2	48	2048	256	16	4096
GP100	Pascal	6.0	48	4096	256	56	14336

- AMD Radeon HD 7970 GPU: 16 KB L1 per computing unit, 768KB shared L2 and a total of **8.25 MB RF**

In Fermi and Kepler, 48KB is the maximum size of L1 cache.

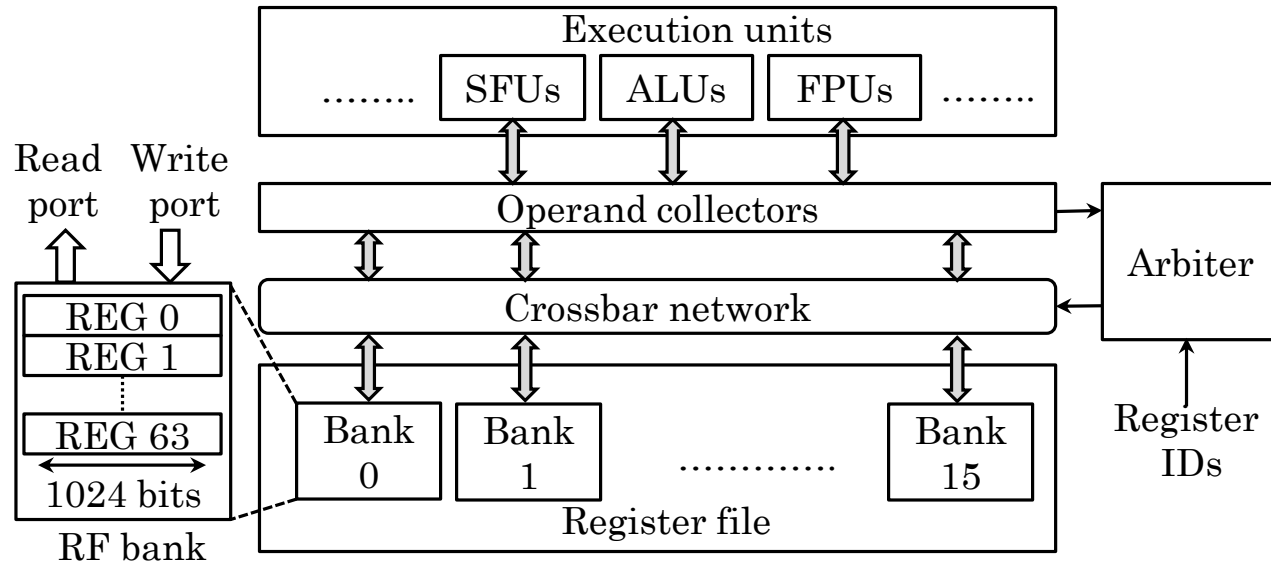
Also, in GM204, 48KB is the size of unified L1/texture cache

RF size in CPU and GPU

- Intel's 32nm Itanium 9560 processor:
 - **32MB L3 cache**
 - **22KB integer RF and 20KB floating point (FP) RF** (1.375KB integer RF and 1.25KB FP RF per thread for 16 threads)

Clearly, GPU RF size is much higher than CPU RF size

GPU RF architecture



Thread register: 32b

Warp register: 1024b ($= 32 \times 32\text{b}$)

Every 1024b RF entry provides 32b operand to all 32 threads in a warp.

Need of RF power management

- Due to large size and high access frequency, GPU RF consumes large power
- 17.2% and 13.4% of total dynamic power in Quadro FX5600 (CC = 1.0, 32KB RF) and GTX 480 (CC = 2.0, 128KB RF) GPUs is from RF
- RF access intensity $\sim 10\text{-}30\times$ L1 cache
- Dynamic power of RF $\sim 6\times$ its leakage power

Challenges in RF management

- RF needs to allow multiple concurrent accesses to provide high throughput
 - e.g., 4 access/cycle to perform fused-multiply-add in one cycle
- Multi-ported RF incurs high overhead => modern GPU RFs use multiple banks and operand collectors
 - Increasing # of banks => diminishing returns
- Need both high capacity and high speed => slow NVMs cannot be used

Challenges in RF management

- Average usage of RF remains low
 - Over-provisioning of capacity to meet peak demands
 - RF bank conflicts
 - Long-latency memory stalls
 - After last read, a register becomes free
- Threats to RF reliability
 - Soft-errors
 - Process variation
 - NBTI (negative bias temperature instability)

Key Ideas of Some Techniques for Managing GPU RF

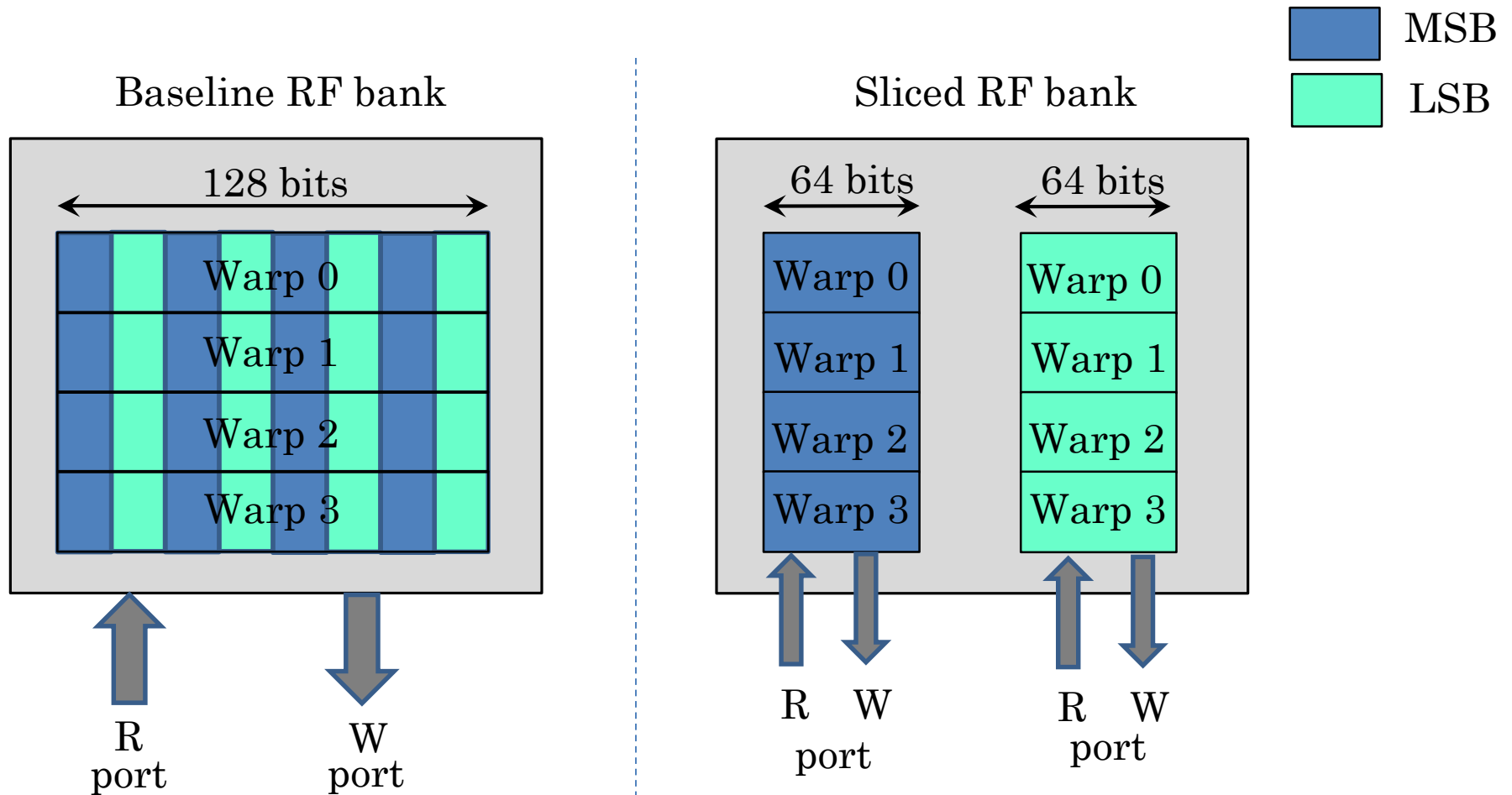
Register File Caching

- Register accesses show high locality
- Use a small "**register cache**" (RegCache) to capture hot registers of *active* threads.
 - Helps in avoiding writing of short-lived values to main RF.
 - Even 6 entry/thread RegCache can reduce accesses to main RF by half!
- Based on register lifetime analysis, do not writeback dead registers from RegCache to main RF

Incremental register allocation

- Baseline: all registers are available for the applications to use.
- **Idea:** Initially, only use selected registers to prioritize their use, e.g., registers in fast banks in 'process variation' affected RF.
- Use remaining registers (e.g., those in slow banks) only if the demand exceeds the available capacity.

Exploiting narrow values



Only LSB slice is accessed for narrow values
=> Energy saving

Exploit narrow values

- Many registers store narrow value, e.g., only 16b data in 32b registers
- Two such writes can share the bus bandwidth

Compression in RF (1/2)

- All threads in a warp execute the same instruction
=> RF access happens at warp granularity.
- Threads in a warp are identified using thread-indices => computations that depend on thread-indices operate on register data that show value similarity.

```
__global__ void kernel(int n, float a, float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```



Thread-index

Compression in RF (2/2)

- => Within a warp, the arithmetic difference between two consecutive thread registers is small
- Perform BDI compression on different thread values
- Use this to save dynamic and leakage energy and also improve soft-error reliability

Avoiding RF bank conflicts

- Issue: RF banks remain idle in some cycles but see conflict in other cycles
- In GPUs, many ready warps remain available at issue stage
- Identify warps to be issued in next cycle
- Find bank conflicts between operands of present and next warp instructions
- Proactively perform upcoming read from an idle bank => avoids potential bank-conflict and pipeline stall

Managing SRAM based RF

- **Issue: SRAM leakage power is high**
- Use state-preserving or state-destroying leakage energy saving techniques, e.g.
- During long-latency operations, place registers of a thread in state-preserving mode
- Power-gate unused registers and registers of inactive threads

Managing SRAM based RF

- Frame processing rate of greater than 30 frames/second (i.e., real-time rendering) does not improve user experience
- Estimate number of threads required for a desired frame rate
- Bring frame rate to 25-30 frames/sec, power-gate extra register banks.

Refresh management in eDRAM RF

- **Baseline: Refresh all banks simultaneously; no data access during this time**
- Schedule refresh operations when a bank is idle to avoid interfering with normal accesses. If no idle period => force a refresh
- Refresh banks sequentially, not simultaneously
- Avoid refreshing dead registers (use compiler)
- Refresh MSB and non-MSB at regular and reduced rate, respectively

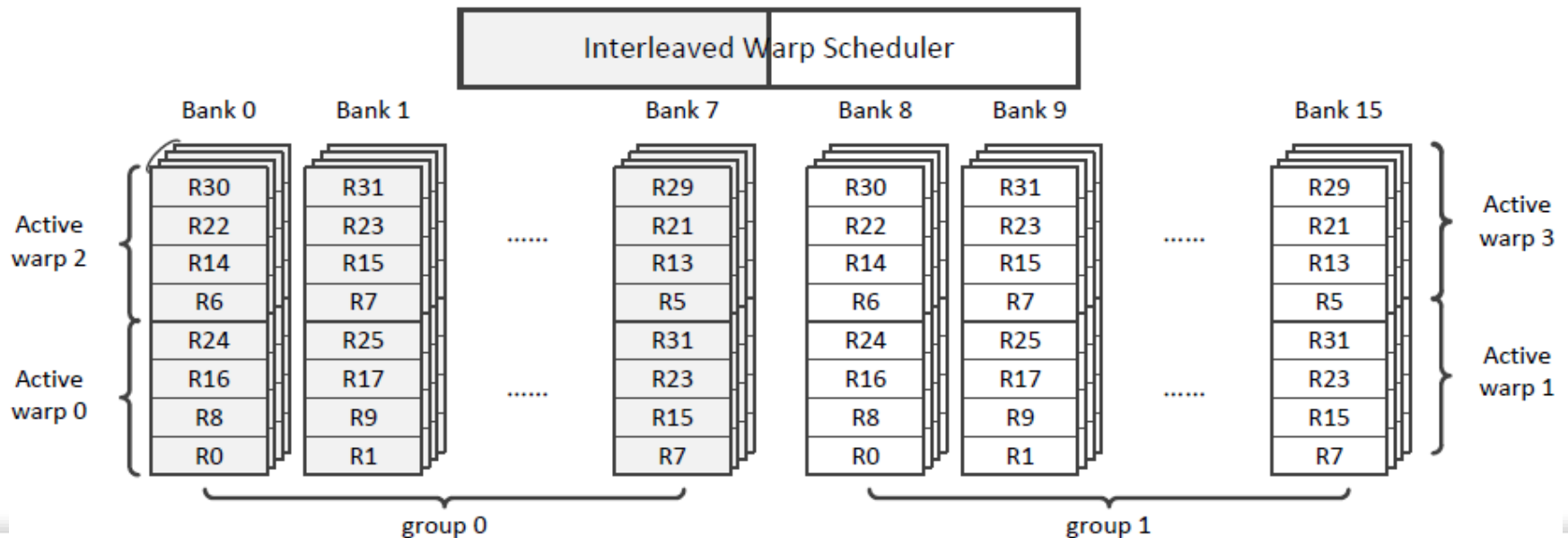
PV management in eDRAM RF

- Profile retention period of different cells
- Refresh banks at different rate, determined by most leaky cell in that bank and not globally

	Bank 0	Bank 1		Bank 15
Retention period of weakest cell	45us	55us		35us
Refresh period in naïve scheme	35us	35us		35us
Refresh period in per-bank refresh scheme	45us	55us		35us

Addressing destructive reads in 1T1C eDRAM (1/2)

- Double # of banks and reduce entries in each bank by half
- Divide warps in odd/even groups based on their ID



Addressing destructive reads in 1T1C eDRAM (2/2)

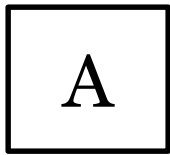
- Issue even/odd warps in successive cycles and map them to even/odd banks, respectively
- When an odd warp accesses odd bank group, read and restore happen in one cycle each
- An even warp is issued to even bank group after odd warp issues
- Bank conflict and destructive read penalty avoided by interleaving

Managing NVM based RF

- Use write-buffers to coalesce accesses
- Use differential write scheme

Bit to write

B



Stored bit

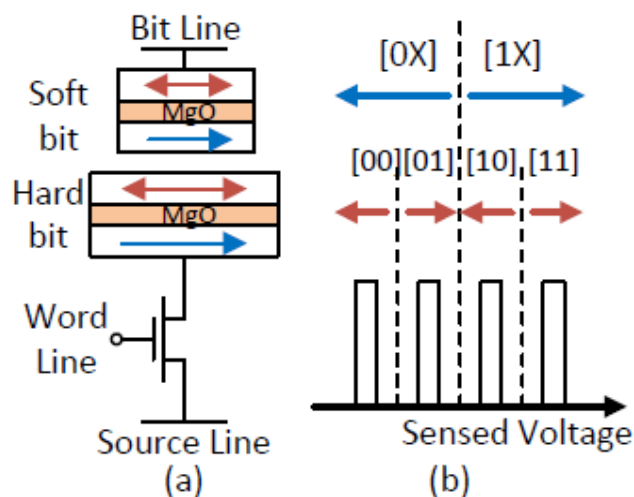
- Before writing
 - read A and compare with B
- If $A=B$, skip write operation

Differential write scheme

Will differential write scheme be useful for SRAM?

Managing MLC STT-RAM based RF (1/2)

- MLC STT-RAM has one hard and one soft bit
- Access latency and energy of hard bits $\gg$ soft bits



	Soft	Hard
Read Lat. (ns)	1.25	1.63
Write Lat. (ns)	7.18	14.86
Read Energy (nJ)	0.018	0.023
Write Energy (nJ)	0.087	0.14

Managing MLC STT-RAM based RF (2/2)

- Design 2048b RF with 1024 MLC cells
 - 1024 hard and 1024 soft-bits (slow/fast row)
- Frequently used registers => soft bits
- Infrequently used registers => hard bits
- Use hard-bit rows only when more than half of total registers are required

SRAM-STTRAM hybrid RF

- Use compiler to find lifetime of registers.
- Most values have short-lifetime and they do not much affect RF soft-error vulnerability
- Remaining (~20%) long-lived values contribute significantly (~90%) to vulnerability
- Map long-lived (i.e., more than 10 instructions) registers to STT-RAM and short-lived to SRAM

Managing DWM-based RF

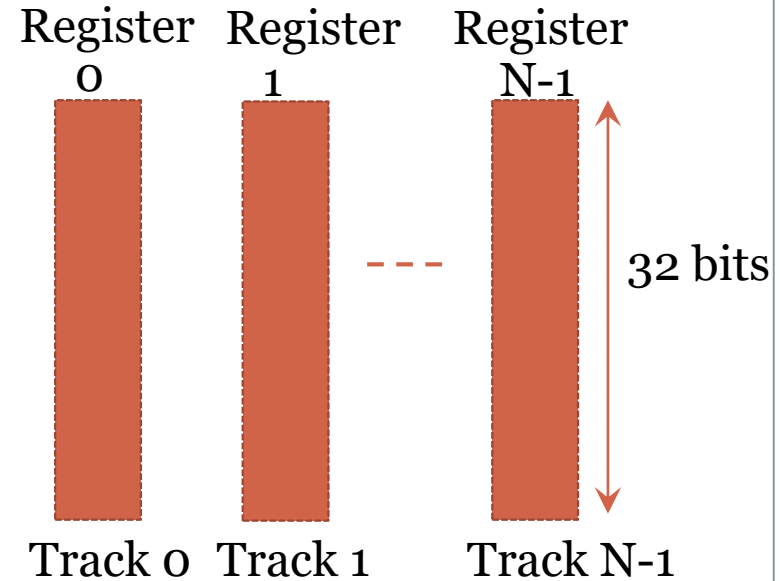
- Predict future access based on past behavior
- Based on this, proactively shift DWM head for hiding shifting latency
- Use bit-interleaving to hide shift latency (see next slide)

Mitigating DWM Shift Overhead

Baseline: one register stored in each track

To access a register, 32b serially accessed from a track
=> very high access latency

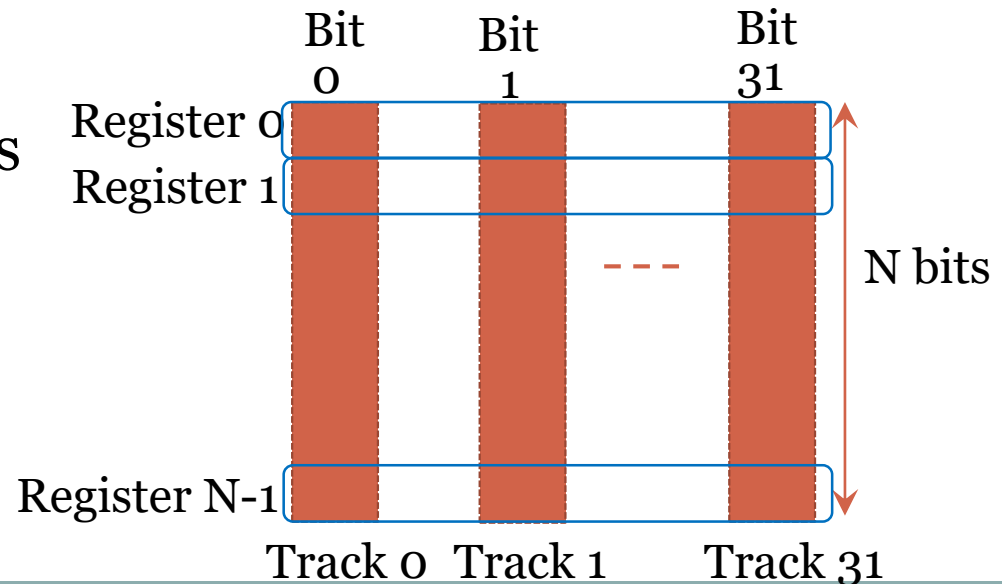
Activate only one track for accessing one register



Bit-interleaved organization:
N registers stored in 32 tracks

All bits of a register can be accessed in parallel

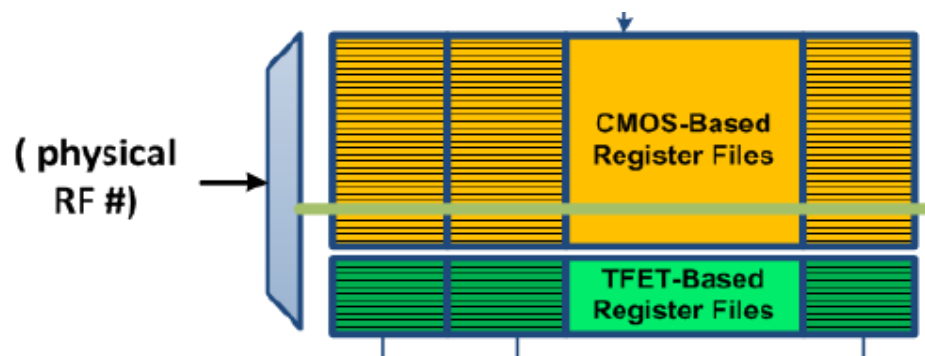
Need to activate 32 tracks for accessing each register



Managing CMOS-TFET hybrid RF

- CMOS and TFET: different transistor types.
- Low voltage: TFET provides higher performance and ultra low leakage than CMOS
- High voltage: CMOS shows better performance

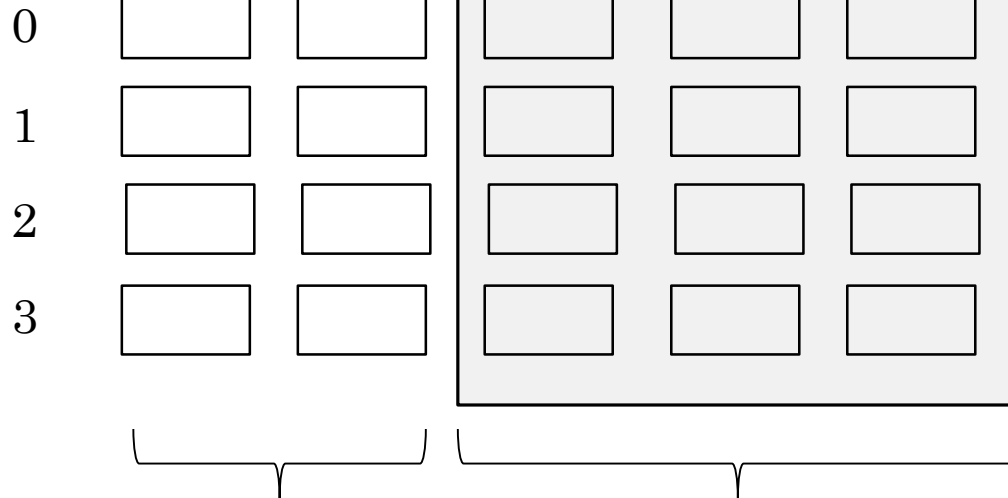
To reduce memory contention, slow-down some threads by mapping their registers to TFET



Improving register utilization (1/2)

- RF size is large, but not all applications or application phases may use all registers
- Share registers across warps

Registers
of thread



Assume 4 threads/warp

Each thread has 5 registers & contributes 3 registers to shared pool

=> A pool of 12 shared registers

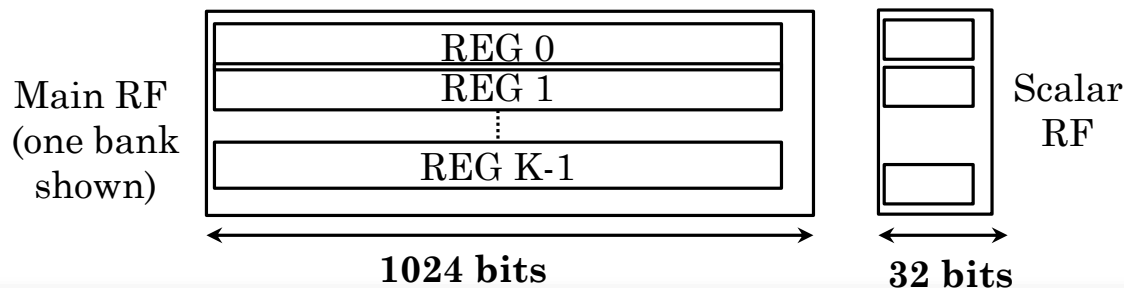
Private Registers Shared Registers

Improving register utilization (2/2)

- Use shared registers for
 - For storing prefetched data
 - As a shared memory pool accessible to all threads
- Storage for cache, RF and shared memory can be unified to just allocate right amount of memory for each of them

Result reuse techniques (1/2)

- Input and output values of different threads in a warp *and* different warps may be same
 - Due to several code features, e.g., loops, initialization from constant values or same address.
- **Intra-warp result reuse:**
 - Only one thread computes, remaining threads of the warp reuse this result
 - Issue such warps to a separate scalar pipeline which uses scalar RF. Reuse one result for all threads



Result reuse techniques (2/2)

- **Inter-warp result reuse:** Use a buffer to store result of warps. If a future warp is found to be present in buffer, reuse the result

Leveraging warp-scheduling schemes

- Preferentially issue ready warps that access unoccupied register banks => to hide RF access latency
- Uniformly spread accesses to registers to reduce NBTI-stress

Precision-aware soft-error protection

- In floating-point intensive GPU apps
 - large magnitude errors can get further amplified to significantly degrade output
 - small magnitude errors have negligible impact on output.
- To cost-effectively reduce large magnitude errors:
 - Store only register MSBs in hardened cells
 - Only verify MSBs of significand computation through redundant execution

References

- S. Mittal, “A Survey of Techniques for Architecting and Managing GPU Register File”, TPDS 2016 ([pdf](#))
- S. Mittal, “A Survey of Techniques for Designing and Managing CPU Register File”, CPE 2016 ([pdf](#))